Chapter 2

REVIEW OF RELATED LITERATURE

This chapter presents an evaluated, organized, and synthesized collection of recent academic literature. The selected journal articles and studies serve as the theoretical foundation for the development of BrewManager. Rather than reviewing technologies one by one, the review is arranged as a problem chain. It first establishes that recorded inventory drifts away from physical stock in small food service operations. It then examines each technical response in turn, showing what each one solves and what it leaves unsolved. This structure locates the specific gap this study addresses: correcting accumulated inventory drift in operations that cannot afford the hardware or staff time that existing solutions assume. The review is organized thematically and aligned to the specific objectives of this study.

Inventory Drift in Small Food Service Operations

Every inventory system assumes its records match what is physically on the shelf. In practice that assumption fails, and it fails gradually. The gap between recorded and physical stock is well documented in the operations literature, where it is called inventory record inaccuracy. It does not come from one large failure. It builds up from many small unrecorded events during normal daily work.

Destro, Staudt, Somensi, and Taboada (2023) modeled this build-up using system dynamics simulation. They found that even a small undetected source of inaccuracy can accumulate over time and eventually destabilize the whole inventory system. Their work also shows why the timing of correction matters. Where stock is verified only through occasional full counts, error keeps accumulating in the records until the next count happens. The gap between counts therefore decides how far records are allowed to drift before being brought back to reality.

The problem is worse for perishable goods. Rekik, Oliva, Glock, and Syntetos (2026) analyzed around twenty-four thousand stock keeping units across eleven grocery stores. They found that inventory record inaccuracy is higher for items that are restocked often, held in larger quantities, and are perishable. Perishable items drift more than durable ones. This applies directly to a coffee shop, where the ingredients that matter most, such as milk, syrups, and dairy alternatives, are exactly the perishable, frequently restocked items the literature identifies as most at risk.

In smaller operations the problem is made worse by how records are kept. Neve and Schmidt (2021) modeled point-of-use inventory systems and showed that recorded and actual stock separate over time because usage is never captured perfectly. They also found that the labor needed to correct this gap becomes a barrier to correcting it at all. Gong and Hui (2024) observed something similar, noting that manual recording introduces calculation errors that

real-time data processing removes, which reduces how clearly a business can see its own operations.

Together these studies establish the problem this study addresses. Records drift from physical reality. The drift builds up rather than settling. Perishable stock drifts fastest. And the work of correcting it is itself an obstacle to getting it corrected. This is the situation independent coffee shops operate in, and it is what BrewManager is designed to address.

Offline-First POS Architectures in Small Retail

Small food businesses operate differently from large chains. They do not have dedicated IT teams or stable network infrastructure, which means the type of point-of-sale system they use is not just a technical choice but an operational one. If the system fails because the internet drops, the entire business stops. This makes the architecture of a POS system a critical concern for small independent shops.

Several studies have examined how POS systems perform in small food business environments. Divina, Olan, Perez, Sarmiento, Acepcion, and Cabututan (2025) developed a web-based POS with integrated forecasting. They found that while the system improved operational speed, they strongly recommended offline capabilities so the system would stay usable during connectivity problems. Natividad, Musngi, Molinyawe, Rivera, Torres, and Moreno (2024) supported this, observing that while small food businesses are open to cloud-based POS systems, their success depends heavily on a stable

internet connection that many independent shops cannot guarantee. On the device level, Sari and Purba (2021) showed that an Android cashier application using a local SQLite database can handle sales and inventory tracking without an external server, making it a realistic and affordable option for small business owners.

These findings point to offline-first architecture, which is the idea that an application should be built to work without the internet rather than simply cope when it is missing. Pothineni (2024) defines this approach as treating network access as optional and relying on local storage to keep the experience consistent regardless of connectivity. Doodala (2023) applied this in an Android application for field operations, using modular caching to prevent data loss when the device went offline. Ramidi (2022) demonstrated the same approach in a mobile health context, using local storage and timestamp versioning to keep data collection running in low-bandwidth environments.

What this literature establishes is that offline operation is already a solved problem. Local storage on a single Android device is enough to run sales and inventory work without a server, and the approach is well understood. BrewManager therefore treats offline-first architecture as a settled design decision rather than a contribution, using a standalone Android application with an embedded SQLite database so that all transactions and ingredient deductions are processed on the device itself. What this architecture does not address is whether the records it keeps stay accurate. An offline system calculates stock just as reliably as an online one, and drifts from physical reality just as steadily.

Automated Recipe-Level Deduction and the Limits of Sales-Based Tracking

If manual record-keeping causes drift, automation is the obvious answer, and the literature consistently identifies it as such. Ugbebor, Adeteye, and Ugbebor (2024) found that automated inventory systems significantly improve stock accuracy and reduce shortages by replacing error-prone manual records with continuous digital updates. Gong and Hui (2024) made a similar argument, stating that moving from manual recording to real-time data processing reduces calculation errors and improves how clearly a business can see its operations. In the food and beverage industry specifically, Geminarqi and Purnomo (2023) identified process automation and intelligent data analysis as key factors in improving efficiency, linking technology adoption directly to better service accuracy. Salih, Ghazi, and Aljanabi (2023) added that automated inventory tracking in medium-sized operations reduces holding costs, though they also noted that structured staff training is needed to move workers away from paper-based habits.

Applied at the ingredient level, this principle produces recipe-based deduction. Each completed sale automatically subtracts the standard amount of every ingredient from recorded stock. This removes manual entry from the transaction entirely and keeps a continuously updated theoretical stock level without staff doing anything.

There is a limit to what this achieves, and it is a limit of principle rather than of how well the code is written. Automated deduction works from recorded sales. It can only account for material that a sale accounted for. Losses that

happen outside a sale are invisible to it, no matter how accurate the deduction logic is. Neve and Schmidt (2021) describe exactly this condition as imperfect usage capture, and show that the resulting gap persists and grows. A deduction engine that subtracts recipe amounts perfectly at every sale will still report figures that become steadily more wrong, because spillage, over-pouring, dropped containers, and spoilage produce no sale to deduct from.

This matters for how automation should be understood. Automated deduction removes one source of drift, which is manual entry error, and leaves another untouched. Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) point to this limit indirectly. For perishable inventory in small and medium enterprises, they recommend continuous physical sensing rather than relying on sales records alone, though they note that the upfront cost of such systems often discourages small operators from adopting them.

BrewManager uses automated recipe-level deduction as the foundation of its inventory tracking. The Automated Recipe Mapping Engine runs at every checkout and calculates theoretical stock with no manual input. The system treats this figure as a theoretical quantity rather than an accurate one, which is a direct response to the limit this literature identifies. It is paired with a physical verification process covered later in this review.

Expiry-Driven Stock Routing in Perishable Inventory

Managing perishable ingredients is one of the highest-stakes parts of running a food service business. When stock rotation is handled poorly, the

consequences go beyond financial loss. Expired ingredients can cause food safety violations, damage a shop's reputation, and in serious cases put customers at risk. The literature consistently shows that manual shelf-life tracking is not reliable enough to prevent these outcomes, and that an algorithmic approach to expiry routing is needed to meaningfully reduce spoilage.

Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) found that digital inventory monitoring in small and medium enterprises significantly improves shelf-life performance and reduces inventory costs. They also noted that the upfront cost of such systems often discourages small operators from adopting them. Herron, Garner, Siddique, Huang, Campbell, Rao, and Morey (2022) argued specifically for First-Expired, First-Out routing over the more traditional First-In, First-Out method. They demonstrated that prioritizing ingredients by expiry date produces much greater reductions in food waste than simply using the oldest delivery first.

This distinction matters in practice. First-In, First-Out assumes that the order stock arrives matches the order it expires. That assumption holds for a single wholesale supplier delivering uniform lots. It does not hold where a shop buys through several channels, since a newer purchase often expires sooner than an older one. Routing by expiration date rather than by arrival date is therefore not a refinement of stock rotation but a correction to it.

The effectiveness of FEFO at the system design level has been demonstrated across multiple studies. Carbajal-Vásquez, Piscoya-Alvites, Quiroz-Flores, García-López, and Nallusamy (2023) integrated FEFO protocols

into a food production facility and reported a 65.57 percent reduction in deteriorated products by explicitly prioritizing items closest to expiration. Hietasari, Subianto, and Adi (2024) proposed warehouse management frameworks that automatically enforce FEFO by dynamically sorting storage locations based on expiry timestamps. Suárez, Guaño, Pérez, and Roa-López (2024) formalized this further through a multi-objective FEFO dispatch model that ensures only ingredients meeting strict quality and time thresholds are used in production. At the restaurant level, Taufiqqurrachman, Nurrohmah, Warno, and Kusumawati (2025) built an Android-based expiration alert system using FEFO methods, showing that food service staff could detect near-expiry items and maintain food safety standards without centralized warehouse infrastructure.

These studies confirm that FEFO routing and proactive shelf-life alerts are proven tools for reducing spoilage. A common limitation across most of these frameworks, however, is that they assume centralized warehouses, IoT hardware, or complex optimization software. None of these reflect the reality of a small independent coffee shop. A second limitation is more fundamental. FEFO governs which batch the system deducts from, but it cannot ensure that staff physically take stock from that batch. The routing rule is enforced in software while the actual withdrawal happens by hand, and the literature reviewed here does not address what happens when the two diverge.

BrewManager adapts these validated FEFO principles into a lightweight offline environment. A Proactive Expiry Monitoring Matrix visually alerts staff when stock is approaching expiration, addressing the food safety goals identified

by Taufiqqurrachman, Nurrohmah, Warno, and Kusumawati (2025). FEFO is enforced at every checkout through a multi-batch splitting algorithm that routes ingredient deductions to the earliest-expiring active batch, with no external hardware or centralized warehouse required. Because the system cannot enforce which container staff physically open, it treats deviation from expiry-order withdrawal as a condition to be detected rather than assumed away, which is addressed in the following section.

Physical Verification and the Correction of Accumulated Drift

The previous sections establish a boundary. Automated deduction cannot see losses that happen outside a sale. Expiry routing cannot check what staff physically take off a shelf. Both work from records. Neither observes the shelf. Closing the gap between recorded and physical stock therefore needs a step that no amount of software logic can provide. Someone has to look.

The operations literature treats this as standard practice. Physical verification takes two broad forms. A full inventory count checks all items at once, usually rarely. Cycle counting checks small groups of items continuously on a rotating schedule. Destro, Staudt, Somensi, and Taboada (2023) compared these through simulation. They found that relying on infrequent full counts lets error build up in the records until the next count corrects it. Cycle counting reduced inaccuracy in every warehouse category they modeled. It also required a dedicated team of counters, and the number needed changed depending on how badly the operation was already performing. Their results discourage using

an annual full count as the only correction measure, while showing that continuous counting carries a real and ongoing labor cost.

The benefit of correction has been measured directly. Rekik, Oliva, Glock, and Syntetos (2026) ran a field quasi-experiment across grocery stores. They found that performing an inventory audit produced an eleven percent store-wide sales lift in the months after the count. The effect was not spread evenly. The entire lift came from items where system inventory was higher than actual inventory, meaning items the system thought were in stock when they were not. The benefit was also stronger for perishable items, which the same study found carry higher inaccuracy to begin with. Physical verification therefore gives the most value in exactly the conditions this study deals with.

Research has also looked at how counting should be organized. DeHoratius, Holzapfel, Kuhn, Mersereau, and Sternbeck (2023) compared several procedures for deciding which items in a retail store should be counted and in what order. Their aim was to detect both inventory record inaccuracy and unknown out-of-stocks. Their work shows that deciding what to count is a design decision with measurable consequences, not simply a matter of counting whatever is convenient.

Discrepancies do not all point the same way, and the difference matters when reading a count. Best, Glock, Grosse, Rekik, and Syntetos (2022) studied positive inventory discrepancies, meaning cases where more stock is physically present than the system records. Using a simulation of a retail store with several error-prone processes, they found that misplacement of items was one of the

main causes. This matters for batch-level tracking. Where stock of the same item is held under separate records, taking material from the wrong record creates a surplus in one and a shortfall in the other, even though nothing was actually lost.

Two limits run through this body of work when it is read against the conditions of a small cafe. The first is labor. Every correction method reviewed assumes staff who can be assigned to counting. Cycle counting needs dedicated counters, and Destro and colleagues quantify how many. A cafe running with one or two staff on shift has no such capacity, and Neve and Schmidt (2021) identify this labor requirement as a barrier to accuracy on its own. The second is hardware. The alternative most of this literature points to is sensing equipment. Maheshwari and colleagues recommend continuous physical sensing for perishable inventory in small and medium enterprises, while noting that its cost discourages the operators who need it most. Ugbebor, Adeteye, and Ugbebor (2024) likewise treat IoT integration as the route to real-time accuracy.

This is the gap this study addresses. The drift problem is documented and measured. Perishable stock is known to drift fastest. Physical verification is known to correct it and to produce measurable benefit. What the literature does not offer is a correction method sized for an operation that can afford neither counting staff nor sensing hardware.

BrewManager answers this with a structured end-of-shift reconciliation built around that constraint. Instead of counting selected items on a rotating

schedule, it checks every active batch at the close of each shift. The interval that Destro and colleagues identify as governing accumulated error is therefore reduced to a single shift. Instead of requiring measurement equipment, it records remaining stock as a count of sealed containers plus a fractional estimate of what is open. This needs no scale, no unit conversion, and no arithmetic from staff. The measured variance is written back to the affected batch record, so each shift starts from a physically counted quantity rather than an estimate that has been drifting since the last correction. Because reconciliation is done per batch rather than per ingredient, offsetting variances of the kind Best and colleagues link to misplacement stay visible, which allows deviation from expiry-order withdrawal to be detected instead of cancelling out.

Deterministic Decision Support in Low-Data Environments

Detecting and correcting drift tells an operator what has already been lost. Preventing loss requires acting on stock before it expires. This is a decision problem, and not all decision support approaches suit the environment of a small cafe. Some rely on large datasets and probabilistic models that learn over time. Others use clearly defined rules and scoring criteria. Where data is limited, decisions must be made quickly, and staff need to understand why a suggestion appeared, the second approach is more appropriate. The literature supports this distinction and validates rule-based scoring in localized recommendation settings.

Gupta, Modgil, Bhattacharyya, and Bose (2021) distinguished AI systems by their underlying logic. They argued that deterministic rule-based methods remain effective and appropriate alongside modern machine learning, particularly where limited data or the need for transparency makes probabilistic models impractical. Van der Waa, Nieuwburg, Cremers, and Neerincx (2021) examined the transparency advantage directly. They found that rule-based explanations in a decision support system let users see clearly which factors drove a recommendation. This increases user trust, which matters in fast-paced retail where staff must act on suggestions quickly without second-guessing the system.

In practical recommendation scenarios, multi-criteria decision-making frameworks have proven reliable for ranking options without opaque models. Çalı and Yilmaz Balaman (2019) used criteria weighting to rank alternatives in the hospitality sector and showed that explicit scoring can handle complex, multi-variable business data. Paradowski, Shekhovtsov, Bączkiewicz, Kizielewicz, and Sałabun (2021) analyzed objective weighting methods in multi-criteria decision support and demonstrated that the choice of weighting directly affects which criteria get prioritized. Their work underlines the importance of selecting a transparent weighting approach suited to the specific business context, and of being able to justify it. Shu, Carrasco, Sánchez-Montañés, and Portela García-Miguel (2024) extended these principles into the food and beverage industry by generating restaurant rankings through explicit demand variables rather than hidden representations.

What these studies show collectively is that rule-based scoring can produce useful, trustworthy decision support without large datasets or heavy computation. What they do not address is the specific decision this study faces, which is not what a customer is most likely to want but what the shop most urgently needs to sell before it spoils. Ranking for spoilage prevention inverts the usual objective of a recommendation system.

BrewManager applies these principles through a localized decision support module that runs a deterministic weighted scoring formula entirely on the tablet. The formula ranks available menu items using two explicit variables: the highest expiry risk among an item's required ingredients, and how often the item has sold over the trailing seven days. The purpose of the module is waste prevention rather than sales maximization. Expiry risk therefore carries the greater weight, while recent popularity serves as a feasibility measure of whether an item is likely to sell within the remaining shelf-life. An urgent ingredient in a rarely ordered drink will not be consumed in time regardless of how urgent it is. Because the logic is fully deterministic, the system behaves predictably and its suggestions can be explained to staff, and each suggestion is presented together with the expiry condition that triggered it. Following Paradowski and colleagues on the significance of weighting choice, the weighting applied is treated as a parameter to be evaluated rather than assumed, and each suggestion and its outcome is recorded so the module's effect on spoilage can be measured.

Cognitive Load and System Evaluation in Operational Settings

The accuracy of any inventory system depends on whether the people using it can complete its procedures correctly. This applies with particular force to a physical audit, which is performed at the end of a shift by staff who are already tired. If the reconciliation workflow is too complicated, too slow, or too mentally demanding, staff will make mistakes, skip steps, or avoid it entirely. When that happens, the accuracy gained through automation is cancelled out by human error at the audit stage. Usability is therefore not a presentation concern in this system but a condition of the correction loop working at all.

Dewi, Dantes, and Divayana (2023) found that poor usability, reflected in low System Usability Scale scores, directly causes more user errors during inventory operations. These errors increase the reconciliation workload and undermine the accuracy that automation is meant to provide. The cognitive workload research adds a further layer. Patel and Naini (2025) conducted a field study in a small restaurant comparing fragmented manual tools against an integrated management platform, and observed significant reductions in both cognitive workload and error rates after staff switched to the integrated system. Kosch, Karolus, Zagermann, Reiterer, Schmidt, and Woźniak (2023) surveyed cognitive workload measurement in human-computer interaction and argued for workload-aware designs that actively support users performing time-sensitive, multi-step tasks. A closing-shift inventory audit is exactly that kind of task.

When systems are designed with usability in mind, outcomes improve measurably. Wulandari, Rahman, Fitriyanto, Dewanto, and Munir (2026)

developed a web-based inventory system evaluated against ISO 25010 quality attributes including functional suitability, reliability, performance efficiency, and usability, and achieved strong SUS scores alongside improved workflow efficiency. Kurniawan, Arrozi, Endika, and Afdhal (2026) applied the SUS framework to a web-based POS system and demonstrated that standardized usability evaluation produces quantifiable evidence of how well a system works, while also revealing differences in how staff in different roles learn and adapt to it.

These evaluation frameworks measure whether a system is usable and whether it meets its functional specification. They do not, on their own, establish whether the figures a system produces are correct. For a system whose central claim concerns inventory accuracy, usability results and functional conformance are necessary but not sufficient evidence. Establishing that claim requires measuring the system's output against independently known quantities.

BrewManager responds to the cognitive load findings of Patel and Naini (2025) and Kosch and colleagues (2023) by reducing the closing audit to two inputs per batch, a count of sealed containers and a fractional estimate of what remains open, with no weighing, unit conversion, or arithmetic required of staff. Each control is initialized at the previous shift's value so that routine closings require only small adjustments. Following the evaluation approaches of Dewi, Dantes, and Divayana (2023) and Wulandari and colleagues (2026), the system is assessed using ISO 25010 and the System Usability Scale. These are supplemented by controlled accuracy testing against known starting quantities

and by a comparison of closing procedure duration before and after deployment, so that the system's accuracy and efficiency claims are supported by measurement rather than by user perception alone.

Synthesis of the Reviewed Literature

Taken in order, the reviewed literature forms a single line of reasoning that leads to the design of BrewManager.

The starting condition comes from Destro, Staudt, Somensi, and Taboada (2023), Rekik, Oliva, Glock, and Syntetos (2026), and Neve and Schmidt (2021). Recorded inventory drifts away from physical stock. The drift builds up rather than settling. Perishable and frequently restocked items drift fastest. And the labor of correcting it is itself a barrier to correction. These are the conditions independent coffee shops work under.

The first response is architectural. Divina, Olan, Perez, Sarmiento, Acepcion, and Cabututan (2025), Natividad, Musngi, Molinyawe, Rivera, Torres, and Moreno (2024), and Sari and Purba (2021) show that internet-dependent systems create unacceptable operational risk for small food businesses, and that an offline Android application with local storage is a practical alternative. Pothineni (2024) sets out the principle. This settles how the system should be built, but not whether its records stay accurate. An offline database drifts just as much as a networked one.

The second response is automation. Ugbebor, Adeteye, and Ugbebor (2024), Gong and Hui (2024), Geminarqi and Purnomo (2023), and Salih, Ghazi, and Aljanabi (2023) show that automated tracking improves accuracy and removes manual entry error. Applied at ingredient level this produces recipe-based deduction, which supports the first objective of this study. It also reaches a limit. Deduction works from recorded sales and cannot account for material lost outside one, a condition Neve and Schmidt (2021) call imperfect usage capture.

The third response is expiry-driven routing. Herron, Garner, Siddique, Huang, Campbell, Rao, and Morey (2022), Carbajal-Vásquez and colleagues (2023), Suárez and colleagues (2024), and Taufiqqurrachman and colleagues (2025) show that FEFO routing measurably reduces spoilage and works without warehouse infrastructure, which supports the second objective. FEFO decides which batch the system deducts from. It cannot control which container staff actually open.

Both limits lead to the same requirement, and it is what the third objective addresses. Correcting drift means observing physical stock, not calculating it. Destro and colleagues (2023) show that the gap between verifications decides how far records drift, and that infrequent full counts let error build up. Rekik and colleagues (2026) measured the benefit of correcting it, finding an eleven percent sales lift concentrated on items where recorded stock was higher than actual stock, with the strongest effect on perishables. DeHoratius, Holzapfel, Kuhn,

Mersereau, and Sternbeck (2023) show that deciding what to count is a consequential design choice. Best, Glock, Grosse, Rekik, and Syntetos (2022) show that discrepancies run in both directions and that misplacement causes positive discrepancy. This is why reconciling per batch rather than per ingredient keeps the signature of misrouted stock visible.

Every correction method in this literature, however, assumes resources a small cafe does not have. Cycle counting needs dedicated counters, and Destro and colleagues quantify how many. The alternative most often recommended is sensing hardware, which Maheshwari, Kamble, Pundir, Belhadi, Ndubisi, and Tiwari (2021) and Ugbebor and colleagues (2024) endorse while admitting its cost puts it out of reach for small operators. This is the gap this study fills: a correction loop sized for an operation with neither counting staff nor sensors. It works by checking every active batch each shift through container counts and fractional estimates, then writing the measured variance back to the batch record.

The fourth objective follows from turning measured expiry risk into action. Gupta, Modgil, Bhattacharyya, and Bose (2021) and van der Waa, Nieuwburg, Cremers, and Neerincx (2021) support deterministic, transparent algorithms over opaque models where interpretability matters. Paradowski and colleagues (2021) show that the choice of weighting materially affects outcomes and therefore needs justification. Combined with the localized multi-criteria approach of Shu and colleagues (2024), the literature supports a rule-based scoring layer

as the right design for a small cafe, with weighting treated as a parameter to be tested rather than assumed.

The fifth objective concerns evidence. Dewi, Dantes, and Divayana (2023) show that poor usability produces inventory errors directly. Patel and Naini (2025) and Kosch and colleagues (2023) establish cognitive load as a design constraint for time-sensitive, multi-step tasks, which the closing audit is. Wulandari and colleagues (2026) and Kurniawan and colleagues (2026) show that ISO 25010 and the System Usability Scale produce quantifiable evaluation evidence. Because these instruments measure usability and functional conformance rather than numerical correctness, this study adds controlled accuracy testing and a before-and-after comparison of closing duration, so the system's central accuracy claim is supported by measurement rather than by what users think of it.

Conceptual Framework

The conceptual framework serves as the structural blueprint for the design and development of the BrewManager system. This study uses the Input-Process-Output (IPO) model to map how data entered by each user role is handled, how the offline SQLite database processes it through deterministic algorithms, and what results are returned to the users.

The framework is arranged to show a correction loop rather than a one-way flow. Sales produce automatic deductions, which generate a theoretical stock level. That level drifts from physical stock because unrecorded losses

occur outside sales. The end-of-shift audit measures the gap, and the correction is written back into the same inventory records the deduction engine reads from. Each cycle therefore begins from a physically counted quantity rather than an estimate carried forward. Figure 2.1 presents the conceptual paradigm of the study, showing the boundaries between barista tasks, manager configuration, the internal logic of the application, and the resulting outputs.

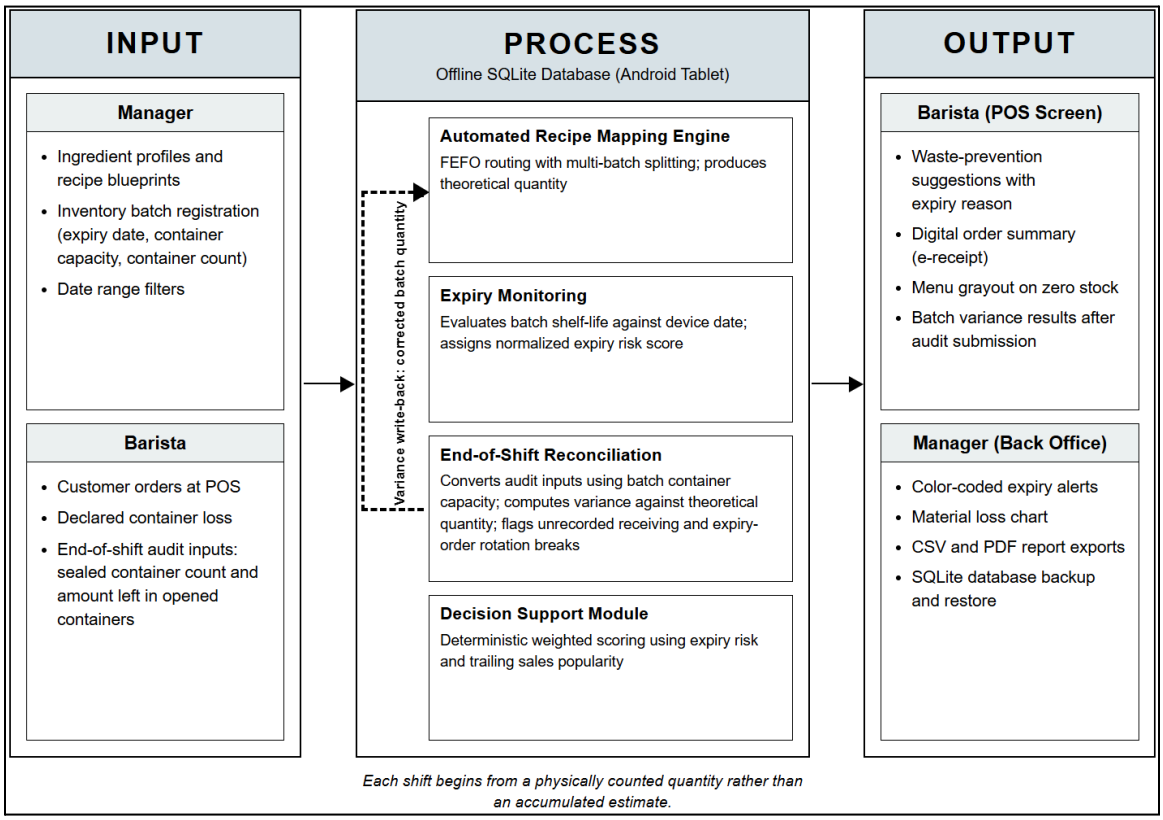


Figure 2.1 Conceptual Framework

Input

The input phase covers all data entered into the system through the Android tablet interface. The PIN-based permission matrix controls what each role can enter, so inputs are divided between the two roles.

Under the Manager role, inputs are mainly configuration and setup data. These include recipe blueprints defining the ingredients required for each beverage, and inventory batch records created at receiving. Each batch record holds one expiration date, one container capacity, and the number of containers received with that date. Because a single purchase can contain containers with different expiry dates or different sizes, one receiving session may create several batch records. The Manager also supplies date range parameters when filtering the analytics dashboard.

Under the Barista role, inputs are operational and occur during the shift. During service, baristas enter customer orders through the POS screen. They may also record a mid-shift loss, such as a dropped container, through the declare container lost function. At the close of the shift, they submit an audit for every active batch. Each batch takes two inputs: a count of sealed, unopened containers, and the combined amount left across all opened containers of that batch, entered in quarter-container steps. No weighing, unit conversion, or arithmetic is required.

Process

The process phase covers the logic the system performs inside the device's local SQLite database. None of these operations require an internet connection.

When a barista completes a checkout, the system triggers the Automated Recipe Mapping Engine. This applies FEFO and multi-batch splitting to deduct the recipe amount of each ingredient from the earliest-expiring active batch,

carrying any remainder to the next batch in expiry order. The result is the theoretical quantity for each batch.

At the same time, the system evaluates each batch against the device clock to determine its shelf-life status and assigns a normalized expiry risk score.

During the end-of-shift audit, the system multiplies the submitted container count and fractional value by the batch's registered container capacity to obtain the audited quantity. It computes the variance against the theoretical quantity and writes the corrected value back to the batch record. This write-back is what closes the loop: the inventory the deduction engine reads at the start of the next shift is the quantity someone physically counted, not an accumulated estimate. The system also checks for two conditions during this process. A submitted count higher than the batch's recorded containers is flagged as possible unrecorded receiving. Two batches of the same ingredient returning variances that cancel each other out are flagged as a likely break in expiry-order rotation.

Separately, the rule-based decision support algorithm runs a deterministic weighted scoring formula. It ranks available menu items using the highest expiry risk score among an item's ingredients and the item's trailing seven-day sales popularity.

Output

The output phase covers what the system presents back to users after processing.

On the barista's POS screen, the system displays ranked waste-prevention suggestions shown together with the expiry condition that

triggered them, on-screen digital order summaries serving as e-receipts, and automatic grayout of any menu item whose required ingredients have reached zero stock. After an audit is submitted, the barista is shown the computed variance for each batch along with any rotation or unrecorded receiving flags.

On the manager's dashboard, the system displays color-coded expiry alerts for all active batches, current stock balances, and a material loss chart drawn from reconciliation logs and filterable by date range and ingredient. The Manager can export sales, inventory, and reconciliation reports as CSV or PDF files for review, and can copy the complete SQLite database file to external storage as a recoverable backup.